Image Tools Hub

Project Structure & Execution Plan

English version prepared for repository documentation

Management decision: build a fast MVP now, but use an architecture that is ready for future backend services, scaling, analytics, paid features, and heavier server-side processing.

1. Project Overview

Image Tools Hub is a web platform for image and file utilities that work quickly inside the browser without requiring desktop software installation. The business goal is to launch a useful SEO-friendly tools platform, attract daily search traffic, and gradually monetize through ads and premium features.

2. Product Direction

- Do not treat the project as a simple front-end demo. It is expected to grow and will likely require backend services later.
- Start with an MVP that is clean, stable, and fast to ship.
- Keep the first release focused on practical image and file tools with strong demand.
- Design everything so client-side tools can later evolve into hybrid or server-side workflows when needed.

3. Core Management Principles

- MVP first: the current target is a working launch, not all 50 tools.
- One consistent interface: all tools should feel like part of the same product.
- Scalable structure: separate UI, feature logic, shared libraries, and future server services.
- Controlled scope: build one tool at a time, or at most two in parallel.
- Backend-ready mindset: prepare for uploads, queues, storage, user history, billing, and analytics.

4. Tool Prioritization

Priority 1 — Must be built first

- Image Resizer
- Image Compressor
- Image Converter
- Crop Image
- Rotate Image
- Flip Image

Priority 2 — Build after the core image layer is stable

- Image to PDF
- PDF to Image
- Resize for Instagram
- QR Code Generator
- Color Picker
- Passport Photo Maker

Priority 3 — Defer until after launch

- Background Remover
- AI Image Enhancer
- PNG to SVG
- Video to GIF
- Bulk tools and other heavy processing tools

5. Official Project Structure

The structure below is the official direction for the repository. It is designed for a simple MVP today and a cleaner migration path to backend-supported features later.

```
image-tools-hub/
├── app/
│   ├── (marketing)/
│   ├── tools/
│   ├── api/
│   ├── layout.tsx
│   └── globals.css
├── components/
│   ├── layout/
│   ├── shared/
│   ├── tool-ui/
│   └── seo/
├── features/
│   ├── image-resizer/
│   ├── image-compressor/
│   ├── image-converter/
│   └── ...
├── lib/
│   ├── image/
│   ├── pdf/
│   ├── utils/
│   ├── constants/
│   └── seo/
├── services/
│   ├── client/
│   └── server/
├── hooks/
├── types/
├── store/
├── public/
├── tests/
├── .env.local
├── next.config.ts
```

```
├── tailwind.config.ts
├── tsconfig.json
└── package.json
```

Why this structure matters

- app: routes, pages, and API endpoints.
- components: reusable UI shared across the product.
- features: tool-specific UI, hooks, and logic.
- lib: shared processing functions and utilities.
- services: future-ready client/server service layer for storage, analytics, auth, and heavy processing.

6. Architecture Rules

- Anything specific to a single tool belongs in features/tool-name.
- Anything shared by multiple tools belongs in components or lib.
- Anything related to future server logic, file persistence, analytics, authentication, or processing pipelines belongs in services.
- Avoid placing complex logic directly inside page components.
- Assume some tools may begin in the browser and later move to server-side or hybrid execution.

7. Execution Roadmap

Phase	Objective	Main Deliverables
Phase 1	Foundation setup	Next.js setup, Tailwind, global layout, homepage, tools index
Phase 2	Core image tools	Resizer, compressor, converter, crop, rotate, flip
Phase 3	Utility tools	Image to PDF, PDF to Image, Instagram resize, QR generator, Color Picker, Passport Photo Maker
Phase 4	Launch polish	SEO basics, metadata, testing, icons, UX improvements, deployment

8. Definition of Done for Each Tool

- User can upload a file without confusion.
- There is a clear preview state before and/or after processing.
- The tool performs the requested transformation correctly.
- The user can download the final result easily.
- The page works well on mobile screens.
- Basic error states are handled cleanly.

9. Suggested Team Split

- Developer A: pages, layout, shared UI components, homepage, tools listing, and UX consistency.
- Developer B: tool logic, file handling, canvas/PDF processing, validation, and test coverage.
- Shared responsibility: naming, code review, architecture discipline, and release quality.

10. Sprint 1 Target

- Create the project foundation using the agreed folder structure.
- Build Navbar, Footer, homepage, and tools index page.
- Complete the first production-quality tool: Image Resizer.
- After that, continue with Image Compressor, then Image Converter.

11. Backend Readiness Notes

- Add app/api from the beginning, even if early tools are mostly client-side.
- Keep the services/server layer ready for future file storage, processing jobs, analytics, and authentication.
- Plan for future needs such as user history, usage limits, subscriptions, and premium tool access.
- Expect some advanced tools to require queues or heavier processing outside the browser.

12. Final Management Decision

The official project strategy is: ship a focused MVP quickly, but build it on a structure that can support backend growth later. The team should avoid overbuilding advanced tools too early and should instead finish the foundation, launch the first high-value tools, and grow from a stable base.